Project Proposal for Team 9 (The ENJIneers)

Development team

- **E**ric Villeneuve 8322907
- **N**ash Lovegren 58748724
- **J**oseph Mondelli 4215230
- **I**an Steyn 12371399

Client: Alwyn Spies, Associate Professor of Teaching Languages

Context: Software Engineering Capstone Project

Project Length: 8 months

1 - Overview

Description

We are proposing a web-based game that aims to improve listening comprehension skills for beginner language students at UBCO, with a particular focus on numbers and locations. With support for different languages and levels of difficulty, the game is a scavenger-hunt style loop. It plays audio clips of locations and room numbers on campus, which players must then find and photograph. Players will receive a score for completing a game, and students will be able to view a scoreboard of students in their course. Instructors will be able to set up games as assignments for students to complete, and view and manage their classes. They will also be able to add new locations and audio clips to the library of available scavenger hunt targets.

Most language learning and practice systems focus on written language, rather than spoken language. This software will help students to practice their listening comprehension of spoken language, and will motivate them to learn by gamifying the process.

The initial project will be developed for use at UBCO, with the option for guest users from other schools or contexts, but we plan to develop it in an extensible way to open up the option of hosting other instances of the game at other universities in the future.

User Profiles

Primary User 1: Guest user

- **Description:** This is any user who is not logged in.
- **What they can do:** Guest users will still be able to start and play the game, as well as receive a score, but will not have an overall saved score or account.
- **Background**: This would typically be high school students, or university students not currently taking a language course. Wide variety of technical ability, but generally expected to be familiar with typical web app interfaces

Primary User 2: Students

- **Description:** Students in introductory language classes.
- **What they can do:** Students will use the game as a practice tool for their classes, or for some form of mark or bonus mark in class. They will be able to log in to an account, join a class, and access their scores and classes, and play the game. They will receive points for completing games, which will correlate to rewards and a position on the class leaderboard.
- **Background**: Wide variety of technical ability, but generally expected to be familiar with typical web app interfaces. Have slightly more time to get to know the system and more experience with learning software.

Primary User 3: Instructors

- **Description:** Instructors are the teachers of the introductory language classes.
- **What they can do:** They will be able to do everything a student can, as well as view their classes, the students in their classes and their scores. Instructors will also be able to create new classes, manage the students in their classes, and export scores for potential grading. Instructors will also be able to add and edit locations, languages and audio clues.
- **Background:** These are not particularly technical users, but will be familiar with concepts of managing students in learning software.

Secondary User: Administrator

- **Description and what they can do:** An administrator is an instructor with elevated permissions, who can edit and delete all game items, view all accounts, and manage instructor permissions of other accounts.
- **Background**: There will likely be only one or two administrators. Still not very technical users, but may have more knowledge of the system and can be taught how to support other users.

Value Proposition

In comparison to other teams working on this project, there are three areas in which we excel and in which we value high quality work.

1. **Fun:** We want to commit to the gamification elements of the app, through both a stylized appearance and a well thought through gameplay loop.
2. **Maintainability and longevity**
    - **Ease of creating and managing accounts/classes:** We want it to be easy to create and manage a class and/or individual accounts, so we will be researching options to integrate with Canvas to make the set-up process as simple as possible.
    - **Ease of adding new elements for instructors and admins:** through clear, well-organized models of data and game items, we want to make it as easy as possible for non-technical users to add new game elements. This will be our philosophy all the way through, from system architecture decisions to UI design.
    - **Maintainability for future developers:** through readable and well-documented code, we want to make it possible for future teams to work on this.
3. **Client relationship and presentation:** We can take ownership of our project and communicate it well to the client and users. We care about making something that our client actually wants to use.

2 - Technology Stack

We will be using Python (FastAPI) as our primary technology for developing the first milestone of this project, and Pytest as our testing framework. See our tables evaluating different options below.

We have also started researching other parts of the stack. We are considering using the follow

- Front-end: Vue.js
- Data: ORM - SQLModel, Database - PostgreSQL, Object Storage - Cloudflare R2
- Canvas Integration: PyLTI1.3

Language/ Backend	**Pros**	**Cons**
Python (FastAPI)	• Built for modern progressive web apps • Good for asynchronous programming and websockets, good for game updates • Allows us to make more architectural decisions than something like Django, so good for learning • Enforces separation of concerns because it serves only data (as JSON), not full webpages/HTML • Uses Pydantic for data validation; much better for debugging • Very fast and high performance, which will be good for the type of live human interaction we're expecting • Can auto-generate API docs, very useful for future maintainers	• Less pre-built stuff than something like Django • Crowded main file • It can be tricky to pass Pydantic validation errors on to somewhere useful
Python (Flask)	• More mature ecosystem than FastAPI, lots of plugins for stuff like login, email, ORM • Interfaces well with Jinja templating if we want to go that route with our front end	• Built for older style of static webpage web-apps, doesn't have as good • No built-in data validation
Stuff that's true for Python in general	• Good for ML/image recognition • Excellent for readability/maintainability • Docstrings make for great function docs	• Weakly typed
JavaScript (NodeJS)	• Very large ecosystem; there are packages and plugins for almost anything • Could potentially use the same language for front and backend • Could be paired with Hono; a super lightweight and easy to use web routing framework	• Less AI/ML options in this ecosystem • No built-in data validation • More bloated and messier code than typical Python applications; more difficult to read and maintain

Testing Framework	Pros	Cons
PyTest (Python)	• Friendly community and very good documentation • Large library of plugins • Very simple to set up and integrate • It shows you the values not just “assertion failed”	• conftest.py sprawled across a big repo can make fixtures hard to find • It's a third-party dependency, not stdlib like unittest
Unittest (Python)	• Part of the Python Standard Library • Widely adopted and well-documented • Consistent test isolation	• Limited plugin architecture • Runs tests sequentially by default • Verbose syntax
Jest (Javascript)	• Compatible with NodeJS, React, VueJS • Standard syntax with documentation support • Allows managing tests w/ larger objects • Very fast and highly performant	• Memory Intensive • Node.js only (not browser native)

3 - Work Breakdown Structure

Functional Requirements	Test Cases	Member
System shall verify GPS places user on campus before allowing gameplay	Simulate GPS coordinates on-campus → game allows start; simulate off-campus coordinates → game blocks start with message	Joseph
Guest can start a game without logging in	Launch game as unauthenticated user → game initializes without login prompt	Eric
User can set difficulty (basic/advancing/challenging) before starting	Select each difficulty level → confirm correct clue type is used	Ian
User can select target language before starting	Select each supported language (Chinese, French, German, Japanese, Korean, Spanish) → confirm audio clips load in selected language	Ian
User can set game length	Set game length to min/max/mid values → confirm number of targets generated matches setting	Ian
System randomly selects 5 locations from a pool of 15–20	Run game start 20+ times → confirm 5 unique locations selected each time, no duplicates within a single game, variation across runs	Ian
System plays audio clue for current target	Load target → confirm correct audio file plays matching language/difficulty	Ian
User can replay audio clue	Trigger replay → confirm same clip plays again without advancing state	Ian
System provides hints after failed attempts	Wait past fail attempt threshold → hint triggers	Nash
User can take/upload photo of location	Submit valid photo → system accepts and evaluates; submit corrupted/invalid file → system rejects gracefully	Eric
System evaluates photo against correct location (image recognition of room number)	Submit photo of correct number → marked correct; submit photo of incorrect number → marked incorrect; submit blurry/unreadable photo → system prompts retry	Eric

User can make multiple attempts per target	Fail attempt 1–2 times, succeed on 3rd → system accepts success and records attempt count	Ian
User can skip target after multiple failed attempts	Fail attempts beyond configured max → system offers/forces skip to next target	Ian
System displays score per item	Complete item → confirm score displayed matches scoring logic (attempts, hints used, time)	Nash
System plays congratulatory audio on success	Successful match → confirm positive-reinforcement audio triggers	Nash
User advances to next item after scoring	Confirm/dismiss score screen → next target loads correctly	Ian
User can pause/resume game	Pause mid-game → timer/state freezes; resume → state continues from same point	Ian
Game saves state after each level (disconnect recovery)	Complete a level, force-close/disconnect app → relaunch → confirm game resumes at correct saved state	Ian
User can quit game	Trigger quit → confirm game session ends and state is handled (saved/discarded per spec)	Ian
System displays total score at game end	Complete all targets → confirm total score matches sum of individual item scores	Nash
System displays game overview post-completion	Complete game → confirm summary view (locations, times, hints used) renders accurately	Nash
(bonus) User can create/join group game via access code	Create group game → generates unique code; join with valid code → success; join with invalid code → error message	Nash
Student can log in to a profile	Valid credentials → login succeeds; invalid credentials → login fails with error	Eric
Student can join a class via access code	Enter valid class code → joins class; enter invalid/expired code → error shown	Eric
Student can leave a class	Trigger leave → student removed from class roster	Eric

Student can view class scoreboard	Open class view → confirm scoreboard reflects current standings for all class members	Nash
Student can view assigned games	Open class view → confirm list of instructor-assigned games with correct settings/due dates	Eric
Student can start an assigned game with preset settings	Launch assigned game → confirm difficulty/language/length match instructor's presets (not user-editable)	Eric
Student can start a custom (guest-style) game	Launch custom game → same behavior as guest game flow (FR-3–FR-6)	Eric
Student can view past individual game scores	Open profile → confirm historical scores list matches completed games	Eric
Student can earn and view achievements/level	Meet achievement criteria → badge/level awarded and visible in profile	Eric
Student can customize profile/UI	Change avatar/theme setting → change persists across sessions	Eric
Instructor can view all their classes	Log in as instructor → confirm all owned classes listed	Joseph
Instructor can create a class	Submit new class form → class created and appears in instructor's list	Joseph
Instructor can generate/share class access code	Create class → unique code generated; share code → student can join using it (ties to FR-24)	Joseph
Instructor can remove students from a class	Remove student → student no longer appears in roster and loses class access	Joseph
Instructor can view student scores	Open class dashboard → confirm scores per student match recorded game data	Joseph
Instructor can export grades	Trigger export → file generated (CSV/etc.) with accurate score data	Nash
Instructor can view past assigned games	Open class → confirm history of assigned games with dates/settings	Nash

Instructor can create an assigned game with difficulty, language, length, due date	Submit assignment form with all fields → game created and visible to students in class before/until due date	Joseph
Instructor can add a new game item/target (location + image + audio clue)	Submit new item with all required fields → item saved and available for game generation	Joseph
System offers selection from existing locations when adding item	Open "add item" form → confirm dropdown/list of existing locations is populated	Joseph
System warns when adding a duplicate location	Attempt to add location matching existing name/coordinates → warning displayed, duplicate not silently created	Joseph
System warns when item already exists for same location+language+difficulty	Attempt duplicate combination → warning shown before save	Joseph
Instructor can upload audio clue file for an item	Upload valid audio file → saved and linked to item; upload invalid/unsupported format → rejected with error	Joseph
Instructor can edit/delete items they created	Edit own item → changes persist; delete own item → item removed; attempt to edit/delete another instructor's item → access denied	Joseph
Admin can edit/delete any game item (not just their own)	Log in as admin → edit/delete item created by another instructor → succeeds	Nash
Admin can view all user accounts	Open admin panel → confirm full account list across all roles	Nash
Admin can grant/revoke instructor permissions	Grant instructor role to a student account → account gains instructor access; revoke → access removed	Nash